

Verifier-Gated LLM NetOps: A Preregistered, Artifact-Backed Evaluation of Input-Sanitization and Deterministic Verification Against Adversarial Intent

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a preregistered paired confirmatory experiment (study_id cand-2026-03) that measured whether template-based input sanitization combined with deterministic verifier-gating (and at-most-one automated repair) changes the rate at which a deterministic validator accepts LLM-generated CLI configurations under adversarially mutated intents. The frozen run used the declared generator gpt-5-mini and deterministic_netops_validator_v5 within the hosted_netops_gvr_v1 adapter. Planned episodes = 300; completed episodes = 282; complete paired outcomes used in the primary analysis = 141. Observed per-pair acceptance: Baseline 135/141 (95.7447%); Guarded 141/141 (100.0%); paired absolute difference = 0.04255319148936165; exact two-sided McNemar $p = 0.03125$; 95% bootstrap percentile CI (10,000 resamples, seed = 1729) = [0.014184397163120567, 0.07801418439716312]. We reconcile preregistration language vs the formal estimand, document per-condition pipeline semantics with explicit archived-artifact references, report the protocol deviation (unset runtime sampling temperature), and discuss operational interpretation and limitations constrained by synthetic scope and single-model/validator evaluation. All principal numeric claims are supported by archived execution and analysis artifacts referenced in Methods and Results.

I. INTRODUCTION

Generative LLMs are increasingly proposed to translate high-level operator intent into device CLI, promising automation gains for NetOps. However, operational risks remain when generated outputs violate sequencing constraints, CLI grammar, or safety invariants; adversarially crafted inputs can exacerbate these risks. Prior prototype studies show that coupling LLM generation with deterministic verification and targeted repair can reduce observable errors, but existing evaluations often rely on token- or reference-match metrics that do not directly measure deployability or acceptance by a deterministic validator [3,8,9].

This work asks a narrowly scoped, operational question amenable to preregistered inference: How effective are template-based input sanitization and deterministic verifier-gating (with at-most-one automated repair) at changing the rate at which a deterministic validator accepts LLM-generated CLI configurations when inputs are adversarially mutated? The study cand-2026-03 preregistered a single formal estimand (paired_success_rate_difference_guarded_minus_baseline) and a confirmatory paired test. Crucially, because the

registered generation semantics were shared_initial_candidate, the paired comparison isolates downstream guarded-pipeline effects (sanitization, gating, permitted repair, and final validation) applied to a single shared LLM draft per intent rather than differences in initial generation.

Contributions. (1) A preregistered, artifact-backed paired experiment executed end-to-end and reconciled against the archived preregistration; (2) a precise, artifact-grounded description of per-condition pipeline semantics that demonstrates shared_initial_candidate reuse; (3) quantitative confirmatory results showing a small but statistically detectable absolute change in deterministic-validator acceptance under the guarded pipeline on a synthetic adversarial-intent bank; and (4) an explicit reconciliation of preregistration natural-language hypothesis wording with the formal estimand and a disclosure of a protocol deviation (unset sampling temperature) together with reproducibility guidance.

II. RELATED WORK

Deterministic network-verification tools (Header Space Analysis, NetPlumber, VeriFlow) provide fast formal checks of forwarding and configuration invariants and are commonly used as validators in LLM-assisted NetOps pipelines [5–7]. Empirical work on LLM-assisted configuration shows that generation alone frequently produces syntactic and semantic mistakes; tool-augmented generate-validate-repair architectures reduce errors in prototype settings [3,8]. Surveys of AIOps and telecommunications identify fragmented evaluation practices and call for operationally meaningful benchmarks that measure deployability and safety rather than token-level similarity [4,9]. Security analyses of LLM-integrated systems document prompt-injection and indirect-prompt attacks that motivate input-sanitization and gating defenses in operational pipelines [10]. Finally, tool-augmented orchestration approaches (Toolformer-style patterns) exemplify how LLMs can call external checks and repair iteratively [11].

Positioning. Unlike many prior studies, we preregistered a single formal paired estimand and exploited shared_initial_candidate semantics so that the observed paired difference strictly reflects post-generation processing (sanitization, deterministic gating, and permitted repair) rather than

differences in initial model sampling. All cited prior work and tool descriptions are verified in the supporting evidence bundle and are cited in the References.

III. METHODOLOGY

Preregistration and estimand (artifact grounding). The study cand-2026-03 was preregistered and archived prior to observing outcomes (preregistration_sha256: ba6ed25e84c7240f94f2d50b3e778fbffd810ff2db632df8946e85f0f1373). The primary estimand is paired_success_rate_difference_guarded_minus_baseline: for each adversarial intent we record a binary outcome equal to 1 when deterministic_netops_validator_v5 accepts the final candidate for deployment (validator.validation_after.valid == true and violation_count == 0 in scoring artifacts) and 0 otherwise. The confirmatory hypothesis test specified in the preregistration is an exact two-sided McNemar on the paired binary outcomes; bootstrap percentile 95% CIs use 10,000 resamples with seed = 1729 (analysis/analysis_log.jsonl).

Design, task bank, and adversary model. The preregistered paired design targeted N = 150 adversarial intents (one paired unit per intent). Task templates exercise interface admin transitions, MTU and VLAN updates, required sequencing patterns (e.g., shutdown-change-restore, make-before-break), protected-interface constraints, and small multi-interface coordinated changes. Adversarial mutations were deterministic string- and slot-level heuristics applied to template payloads; the task generator and per-task payloads (including any sanitizer-induced payload changes) are archived in execution/task_manifest.jsonl (task generator id deterministic_netops_task_generator_v5). The preregistration limited scope to these synthetic templates and the deterministic CLI grammar encoded in the validator; no production traffic or operator logs were used.

Model, validator, and orchestration (explicit recorded configuration). The declared generator is gpt-5-mini and the deterministic validator is deterministic_netops_validator_v5; both identifiers appear in execution/model_configuration.json and in per-episode scoring artifacts (e.g., execution/scoring/task-000001-baseline.json). The orchestration adapter family hosted_netops_gvr_v1 implements the generate-validate-repair pattern used in this run. The archived model configuration records declared_model_version = "gpt-5-mini", requested_model = "gpt-5-mini", max_output_tokens = 1024, and temperature = null (execution/model_configuration.json). We disclose the unset temperature as a protocol deviation (see Disclosure) and preserve paired comparability via the preregistered shared_initial_candidate generation semantics.

Per-pair pipeline semantics and explicit artifact-backed timeline. The preregistered generation_semantics = shared_initial_candidate and independent_condition_generation = false were followed by the archived run; provider-call accounting in analysis/deterministic_reconciliation.json shows stage_counts.shared_initial_generation = 150 and stage_counts.repair = 6, and hosted_model_call_event_count

= 156 (completed_hosted_model_call_event_count = 147, failed_hosted_model_call_event_count = 9). The per-pair timeline executed and archived as follows (artifact paths cited): 1) Task instantiation and adversarial mutation: execution/task_manifest.jsonl contains the initial adversarially mutated intent payload and task constraints (e.g., allowed_touched_fields, workflow_policies). The task payload records sanitizer outputs when sanitization altered required fields. 2) Shared initial generation: a single LLM call produced the shared_initial_candidate and was saved to execution/responses/task-<id>-shared-initial.txt. Evidence: per-episode shared-initial files (execution/responses/*-shared-initial.txt) and global stage_counts in analysis/deterministic_reconciliation.json. 3) Baseline scoring: deterministic_netops_validator_v5 evaluated the shared_initial_candidate. Baseline scoring artifacts (execution/scoring/task-<id>-baseline.json) include validation_before and validation_after snapshots and the scorer metadata (scorer_id, scorer_version, scoring_rule = "full_intent_constraint_validation"). Representative examples: execution/scoring/task-000001-baseline.json, execution/scoring/task-000002-baseline.json. 4) Guarded branch processing: the guarded branch applies a template-based input-sanitizer prior to gating. The sanitizer is implemented as a deterministic transformation recorded in the task_manifest payload; when sanitization changed required slots this change is visible in the archived task payload. The verifier-gate evaluates the sanitized candidate. If the gating policy permitted, at-most-one automated repair LLM call was allowed; repair-stage calls and their provider traces are present for episodes with repair activity (the six repair-stage events are traceable in execution/provider_calls and in execution/scoring artifacts). Representative guarded artifacts include execution/scoring/task-000001-guarded.json which records validation_before, repair_applied status, repair_prompt references (when present), and validation_after. 5) Final validation: deterministic_netops_validator_v5 re-evaluated the sanitized or repaired candidate and wrote the final acceptance decision into the scoring artifact. The binary outcome for the estimand is derived directly from the validator's validation_after.valid field and violation_count == 0.

Operationalization of success/failure, scoring rules, and deterministic checks. The scoring rule used for the primary outcome is full_intent_constraint_validation implemented by deterministic_netops_validator_v5; this validator enforces required-sequence constraints, allowed/touched field restrictions, protected-interface invariants, and CLI grammar. A successful outcome for a condition is defined as the validator marking the final candidate valid with zero violations (as visible in the validation_after object in execution/scoring/*_json). The paired-contingency counts are taken from analysis/paired_contingency_table.csv where rows and columns correspond to guarded and baseline binary outcomes respectively.

Missingness, technical failures, exclusions, and preregis-

tered handling. The preregistered missingness_plan excluded a paired unit only when both paired runs were technical failures (model API failure, verifier timeout, or parser error). The execution manifest reports planned_episode_count = 300, completed_episode_count = 282, and failed_episode_count = 18 (execution/execution_manifest.json). The analysis missingness summary (analysis/missingness_summary.csv) records complete_pairs = 141 and both_missing_pairs = 9, consistent with the preregistered exclusion rule. Provider-call accounting in analysis/deterministic_reconciliation.json documents hosted_model_call_event_count = 156, completed = 147, failed = 9, and stage_counts (shared_initial_generation = 150, repair = 6), which together explain the observed missingness and repair incidence.

Confirmatory and supplementary analysis procedures (archive-referenced). The primary confirmatory test is an exact two-sided McNemar on the paired binary outcomes; this test and the paired point estimate (guarded - baseline) are computed by the archived paired_binary_analysis_v1 executor (analysis/results.json). Confidence intervals are bootstrap-percentile with 10,000 resamples and seed = 1729; bootstrap parameters and the completed-analysis log are archived in analysis/analysis_log.jsonl. Discordant-case diagnostics (n10 and n01 pairs) were inspected by tracing per-episode scoring artifacts referenced in analysis/paired_contingency_table.csv and execution/scoring/*.json to determine whether repairs or sanitization accounted for acceptance changes; the six discordant n10 pairs are associated with repair-stage activity recorded in execution/provider_calls and scoring artifacts.

Preplanned secondary/sensitivity checks and unexecuted items. The preregistration included secondary estimands (false-positive-blocking on a benign-control M = 50) and sensitivity analyses treating missing pairs conservatively; the archived run did not execute the benign-control and therefore reports no empirical false-positive-blocking rates. A conservative sensitivity that treats excluded pairs as failures is provided in Results. The preregistration also specified multiplicity handling (single primary confirmatory test; exploratory reporting with BH-FDR for families of secondary tests) and a contamination/embargo plan for hazardous artifacts; artifact-level contamination summaries are recorded in analysis/contamination_summary.csv.

Reproducibility notes (artifact pointers and protocol deviation). Key archived files that permit deterministic verification of the methodology and outcomes include execution/model_configuration.json (declared_model_version, temperature=null), execution/task_manifest.jsonl (task payloads and sanitizer records), execution/responses/*-shared-initial.txt (shared initial candidates), execution/scoring/*.json (validator traces and final decisions), analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/analysis_log.jsonl, analysis/deterministic_reconciliation.json, and execution/execution_manifest.json. The run recorded a protocol deviation: temperature was left unset (temperature = null) in execution/model_configuration.json. We discuss implications

of this unset sampling parameter in Disclosure and Results and recommend explicitly setting temperature = 0 for fully deterministic reproductions; the shared_initial_candidate semantics and provider-call accounting (stage_counts and hosted_model_call_event_count) demonstrate that the same initial generation was reused across paired conditions, preserving the preregistered paired comparability despite the unset field.

IV. RESULTS

Execution summary and primary counts. The archived execution manifest reports planned_episodes = 300, completed_episodes = 282, failed_episodes = 18 and model_calls_used = 156 (execution/execution_manifest.json). After applying the preregistered paired exclusion rule (exclude pair only when both paired runs are technical failures) the confirmatory analysis used complete_pairs = 141 (analysis/missingness_summary.csv). Condition-level totals used in the primary estimand are recorded in analysis/condition_summary.csv and reproduced here: baseline_successes = 135, baseline_failures = 6 (observed = 141), guarded_successes = 141, guarded_failures = 0 (observed = 141). The condition summary artifact path: analysis/condition_summary.csv.

Paired contingency, test, and interval. The paired contingency table archived at analysis/paired_contingency_table.csv is n11 = 135 (guarded=1, baseline=1), n10 = 6 (guarded=1, baseline=0), n01 = 0 (guarded=0, baseline=1), n00 = 0 (guarded=0, baseline=0). Using the preregistered exact two-sided McNemar test on these paired outcomes yields $p = 0.03125$ (confirmatory result stored in analysis/results.json and summarized in analysis/paired_contingency_table.csv). The observed paired point estimate (guarded - baseline) = 0.04255319148936165. The 95% bootstrap percentile confidence interval (method and parameters preregistered; bootstrap_resamples = 10,000; bootstrap_seed = 1729) is [0.014184397163120567, 0.07801418439716312] (see analysis/analysis_log.jsonl and analysis/paired_difference_figure.svg for bootstrap metadata and visualization). These exact numeric artifacts are archived at analysis/paired_contingency_table.csv and analysis/analysis_log.jsonl.

Discordant-case diagnostics and repair-stage association. All six discordant pairs (n10) are traceable in the archive. Provider-call accounting in analysis/deterministic_reconciliation.json records stage_counts: shared_initial_generation = 150 and repair = 6; hosted_model_call_event_count = 156 (completed = 147, failed = 9). The one-to-one correspondence between the repair-stage call count and n10 suggests the discordant conversions were associated with episodes where the guarded pipeline either applied sanitization that altered the candidate prior to gating or invoked the single allowed automated repair call. Per-episode scoring artifacts for discordant pairs contain validator before/after snapshots (validation_before and validation_after) and, when present,

repair-related provider-trace references (example artifact group: `execution/scoring/task-00000X-baseline.json` and `execution/scoring/task-00000X-guarded.json`). Representative artifacts demonstrating the `shared_initial_candidate` reuse and validator snapshots include `execution/responses/task-000001-shared-initial.txt`, `execution/scoring/task-000001-baseline.json`, and `execution/scoring/task-000001-guarded.json`.

Mechanism observed in archived traces. The archived validator traces show two mechanistic pathways that yielded n10 discordances: (A) template-based sanitizer canonicalized or repaired malformed slots so the sanitized candidate passed the validator when the raw `shared_initial_candidate` had previously failed; or (B) the guarded policy permitted one automated repair LLM call whose output, after re-validation, satisfied the validator. The archive documents both sanitizer activity (task payloads in `execution/task_manifest.jsonl`) and repair-stage call traces where present (provider traces referenced from scoring artifacts). The per-episode scoring artifacts include `validation_before` and `validation_after` objects which record the validator’s `normalized_configuration`, `violation_count`, and `validator_id = deterministic_netops_validator_v5`; these fields permit exact forensic comparison of pre- and post-guarded candidates.

Missingness sensitivity. Nine paired units were excluded because both runs were technical failures (`analysis/missingness_summary.csv`: `both_missing_pairs = 9`). Applying the conservative sensitivity that treats excluded pairs as failures yields `baseline_acceptance = 135/150 = 0.9000` and `guarded_acceptance = 141/150 = 0.9400` (reported in Methods and briefly summarized here as a conservative bound). The archived missingness artifact path is `analysis/missingness_summary.csv`; the reconciliation artifact that records completed/failed provider calls and stage counts is `analysis/deterministic_reconciliation.json`.

Provider-call audit and reproducibility-relevant details. Provider-call accounting (`analysis/deterministic_reconciliation.json`) shows `hosted_model_call_event_count = 156`, `completed_hosted_model_call_event_count = 147`, `failed_hosted_model_call_event_count = 9`, `cache_hit_count = 0`, `cache_miss_count = 147`, and `stage_counts`: `shared_initial_generation = 150`, `repair = 6`. The model configuration artifact (`execution/model_configuration.json`) records `declared_model_version = "gpt-5-mini"` and `temperature = null`; this unset temperature value is a protocol deviation discussed in Methods and Disclosure. The provider-call audit plus the `shared_initial_generation` count provide artifact-grounded evidence that the per-pair generation semantics used a single shared initial candidate (see `execution/responses/*-shared-initial.txt`) and that at most one repair-stage call occurred for the small subset of episodes that needed it. Paths: `analysis/deterministic_reconciliation.json` and `execution/model_configuration.json`.

Representative per-task artifact inspection. For reproducibility and reader verification we point to representative per-task artifact groups archived in

`execution/` (examples listed in the evidence bundle):
- `task-000001`: `execution/responses/task-000001-shared-initial.txt`, `execution/scoring/task-000001-baseline.json`, `execution/scoring/task-000001-guarded.json`;
- `task-000002..task-000006`: analogous `shared-initial`, `baseline`, and `guarded` response and scoring artifacts (see `execution/responses/` and `execution/scoring/`). These representative artifacts show identical `shared_initial_candidate` content referenced by both baseline and guarded scoring records along with validator `validation_before/validation_after` snapshots and the `scorer_id = deterministic_netops_validator_v5`, enabling exact verification of the reported paired outcomes.

Unexecuted preregistered items and limitations relevant to results. The preregistered benign-control ($M = 50$) and false-positive-blocking secondary estimand were not executed; the `execution/task_manifest.jsonl` and responses directories contain no benign-control artifacts. Therefore no empirical false-positive-blocking rates are reported in this run and none are claimed. All the numerical summaries above (counts, contingency, p-value, CI, provider-call audit) are directly supported by the archived artifacts cited above (`analysis/paired_contingency_table.csv`; `analysis/condition_summary.csv`; `analysis/missingness_summary.csv`; `analysis/deterministic_reconciliation.json`; `execution/responses/*-shared-initial.txt`; `execution/scoring/*.json`).

V. DISCUSSION

Hypothesis reconciliation and interpretive boundary. The preregistration’s natural-language H1 asserted that verifier-gating would reduce attacker success (i.e., fewer unsafe configurations accepted). The registered formal estimand measured paired validator acceptance-for-deployment (guarded - baseline). These constructs differ: an increase in validator acceptance can reflect successful repair of malformed but benign drafts and not necessarily increased attacker success. Our observed guarded→higher acceptance (≈ 4.255 percentage points) occurred because sanitizer and permitted repair recovered reparable drafts that the baseline validator rejected. Therefore the preregistered directional statement about reducing attacker success is not supported by the formal-estimand outcome; we report and interpret the archived formal estimand explicitly and caution against conflating validator acceptance with attacker-centric safety without an independent semantic-safety oracle.

Operational implications and repair-gate co-design. Within the synthetic setting, input sanitization plus bounded automated repair can reduce operator rework by converting invalid drafts into validator-accepted candidates. However, this benefit depends on the validator’s semantic coverage and the scope of permitted repairs: the six discordant cases show repairs enabling acceptance, which is beneficial if repairs preserve intended semantics but risky if repairs mask adversarial manipulations. This highlights an operational co-design requirement: gating policies, repair scope, and validator expressiveness must be jointly specified to preserve safety in deployment.

Practical trade-offs illustrated by the archive. The artifact evidence (analysis/deterministic_reconciliation.json, analysis/paired_contingency_table.csv, and per-episode scoring artifacts such as execution/scoring/task-000001-guarded.json) shows precisely how small numbers of permitted repairs materially changed final acceptance. That pattern exposes a predictable trade-off: stricter gating (reject-on-violation) reduces the chance of a repaired but semantically altered candidate being accepted, at the cost of more operator interventions; more permissive policies with bounded repairs reduce operator workload but raise the need for stronger semantic checks or human review on repaired cases. These trade-offs are operational, measurable with the archived logs (repair_stage_count = 6, hosted_model_call_event_count = 156) and must be chosen to reflect acceptable risk budgets in deployment.

Failure modes of generate-validate-repair loops. The archived validator traces document three related failure modes relevant to operational design: (1) reparable syntactic/ordering errors where repair restores intent-preserving structure (the desirable class observed for the six discordants); (2) repair that fills missing fields or normalizes values in ways that change semantics subtly (requires semantic-safety or reachability checks beyond the current validator); and (3) cases of technical failure or timeouts (18/300 episodes, both-run failures = 9) where neither repair nor gating completed, reducing effective sample size (analysis/missingness_summary.csv). Operational pipelines should instrument and surface these distinct failure categories to operators and governance audits using the same per-episode trace primitives present in our archive.

Reproducibility and the protocol deviation. The unset runtime sampling temperature (temperature = null in execution/model_configuration.json) is a recorded protocol deviation. Its practical implication depends on provider default behavior; because the run used shared_initial_candidate semantics (stage_counts.shared_initial_generation = 150) and provider-call accounting confirms the observed call pattern (analysis/deterministic_reconciliation.json), paired comparability for the formal estimand is preserved in this execution. Nevertheless, for deterministic reproduction we recommend setting temperature = 0 explicitly (execution/model_configuration.json) and re-running the archived task manifest; the archive provides all artifacts necessary to compare outcomes deterministically (execution/responses/*, execution/scoring/*, analysis/*).

Governance, auditing, and measurement recommendations grounded in archived evidence. The run demonstrates the value of (a) detailed per-episode validator traces (validation_before/after objects in scoring artifacts) to audit repairs; (b) explicit repair audit logs and provenance (present for repaired episodes) so that operator review can target only changed cases; and (c) a preregistered benign-control to quantify false-positive-blocking (not executed here). The archive’s contamination and missingness summaries (analysis/contamination_summary.csv is empty; analysis/missingness_summary.csv reports counts) show these governance metrics can be computed automatically from the same

instrumentation used in this study.

Limits to operational generalization. The discussion above is tethered to the archive: a single LLM (declared gpt-5-mini), a single deterministic validator implementation, and synthetic templates. These constraints mean the measured acceptance-rate change is a within-bundle operational observation, not a claim of cross-vendor or production robustness. The ceiling effect (high baseline acceptance) reduces practical headroom for improvement; this is visible in analysis/condition_summary.csv and motivates future runs that (1) execute the preregistered benign-control to measure false positives, (2) adopt attacker-centric semantic-safety or reachability oracles in addition to syntactic validators, and (3) diversify model and validator families before making operational deployment claims.

Concluding interpretive note. The archived data concretely show that bounded verifier-gated repair can increase validator acceptance by recovering reparable drafts; whether that outcome improves or harms security depends on whether repairs preserve intended semantics—a property not fully covered by the present deterministic validator and therefore not directly measured here. The empirical artifacts (per-episode scoring traces, repair counts, and paired contingency outputs) provide the necessary operational observables to design follow-up confirmations that target attacker-centric estimands and benign-control measurements.

VI. LIMITATIONS

- Synthetic task bank and intent templates — not real production traffic or operator logs; results may not generalize to field datasets.
- Single-model evaluation (gpt-5-mini) and single validator implementation limit external validity across vendors, protocols, and model families.
- Validator coverage constrained to encoded safety properties (CLI grammar, required sequences, protected-interface invariants); some protocol-level or dynamic runtime semantics are not represented.
- Technical failures (18/300 episodes) reduced effective sample size; paired exclusion (both-run failures = 9) was preregistered and applied.
- Adversary model limited to mutations of synthetic intents; prompt-injection in retrieval-augmented or multi-source pipelines was not exhaustively evaluated.
- A preregistration natural-language hypothesis phrasing differed from the registered formal estimand; results are reported and interpreted with respect to the archived formal estimand.
- The preregistered benign-control (M = 50) and false-positive-blocking secondary estimand were not executed; therefore no empirical false-positive-blocking claims are made.

VII. CONCLUSION

A preregistered, artifact-backed paired experiment on a synthetic adversarial intent bank found that template-based

input sanitization combined with deterministic verifier-gating and at-most-one automated repair produced a small but statistically detectable absolute increase in deterministic-validator-accepted LLM-generated configurations under the archived formal estimand (paired difference ≈ 0.04255 ; $p = 0.03125$; 95% CI [0.01418, 0.07801]). Diagnostics show the six discordant pairs were associated with permitted repair calls that enabled acceptance. Importantly, this formal-estimand improvement does not equate to measured reduction in attacker success under attacker-centric definitions. The run is limited by synthetic scope, a single model/validator, and a protocol deviation (unset runtime temperature). Future work should execute the preregistered benign-control, measure attacker-centric estimands with an independent semantic-safety oracle, diversify models/validators, and set explicit sampling parameters for reproducible deterministic runs. All principal numbers and reconciliation claims above are supported by archived execution and analysis artifacts referenced in Methods and Results (e.g., `execution/task_manifest.jsonl`; `execution/responses/task-000001-shared-initial.txt`; `execution/scoring/task-000001-baseline.json`; `execution/scoring/task-000001-guarded.json`; `analysis/paired_contingency_table.csv`; `analysis/condition_summary.csv`).

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role and autonomy boundary: a human Research Director selected the topic family (Generative AI / LLMs for NetOps) and constrained the initial master prompt prior to the autonomy lock; all literature review, preregistration, study selection, code generation, experiment execution, analysis, peer-review cycles, manuscript drafting, and revision reported here were performed autonomously after the lock. Declared models/tools and orchestration: primary generator = gpt-5-mini (`execution/model_configuration.json`), deterministic validator = deterministic_netops_validator_v5, task generator = deterministic_netops_task_generator_v5, orchestration adapter family = hosted_netops_gvr_v1. Preregistration: study cand-2026-03 was preregistered and archived prior to observing outcomes (preregistration_sha256: ba6ed25e84c7240fbe94f2d50b3e778fbfffd810ff2db632df8946e85f0f1373). Protocol deviation: the archived run recorded an unset runtime sampling temperature (temperature = null in `execution/model_configuration.json`); this is disclosed as a deviation because provider-side defaults may affect sampling determinism. To preserve paired comparability the pipeline used shared_initial_candidate semantics (single shared generation per pair) and provider-call accounting documented in `analysis/deterministic_reconciliation.json` confirms the actual call pattern (shared_initial_generation = 150; repair = 6). Key archived artifact references (examples): `execution/task_manifest.jsonl`; `execution/responses/task-`

`000001-shared-initial.txt`; `execution/scoring/task-000001-baseline.json`; `execution/scoring/task-000001-guarded.json`; `analysis/deterministic_reconciliation.json`; `analysis/paired_contingency_table.csv`; `analysis/condition_summary.csv`; `analysis/analysis_log.jsonl`; `execution/model_configuration.json`; `execution/execution_manifest.json`. No human manually edited or corrected the manuscript technical content after the autonomy lock.

REFERENCES

- [1] <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659>
- [2] <https://doi.org/10.1145/2342441.2342452>
- [3] <https://doi.org/10.1145/3626111.3628194>
- [4] <https://doi.org/10.1145/3605764.3623985>
- [5] <http://arxiv.org/abs/2302.04761>
- [6] <https://doi.org/10.1145/3656296>
- [7] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447
- [8] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635
- [9] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194
- [10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, Mario Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", 2023, 10.1145/3605764.3623985
- [11] Yupeng Chang, Xu Wang, Jindong Wang, Yuan-Hsuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, Xing Xie, "A Survey on Evaluation of Large Language Models", 2023, 10.48550/arxiv.2307.03109
- [12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Răileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, Thomas Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools", 2023, 10.48550/arxiv.2302.04761
- [13] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
- [14] Peyman Kazemian, George Varghese, Nick McKeown, "Header space analysis: static checking for networks", 2012